The Journal of Open Source Software

FMU4cpp: A GitHub template repository for creating Functional Mockup Units in C++

Lars Ivar Hatledal [1]

1 Norwegian University of Science and Technology, Department of ICT and Natural Sciences, Norway

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 December 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

FMU4cpp is a GitHub template repository that acts as a framework for creating Functional Mockup Units (FMUs) compatible with the Functional Mockup Interface (FMI) standard (Blochwitz et al., 2011) in idiomatic C++. The FMI is an open standard for model exchange and co-simulation of dynamic systems. Currently, version 2.0 for Co-simulation (Blochwitz et al., 2012) is supported. In short, an FMU (Functional Mock-up Unit) is a self-contained component that implements the FMI standard. It is packaged as a zip archive containing: - A shared library for each supported platform, which implements a standardized C interface. - A *modelDescription.xml* file, describing the FMU's capabilities and available variables. - Optionally, component-specific resources embedded within the archive.

The intention of FMU4cpp is to provide a streamlined way of generating such FMUs based on C++ source code. To do so, the framework provides skeleton code for the user to implement, so that many of the low level details of the FMI standard is abstracted away. Furthermore, it handles generating the required XML as well as packaging the model according to the requirements. To further simplify model creation, the framework utilizes GitHub Actions in order to automatically build and package cross-platform FMUs. When the users push their changes to GitHub, the model is verified against standard conformance and built on an Ubuntu and Windows runner. The resulting shared libraries are then joined into a final archive which can be directly downloaded and imported into a simulation tool.

Figure 1 demonstrates simulation of a simple *BouncingBall* model generated with FMU4cpp within the Python simulation library FMPy (CATIA Systems, 2017). Similarly, the generated FMU can be imported into any of the many available simulation tools compatible with FMI 2.0 for Co-simulation.

DRAFT

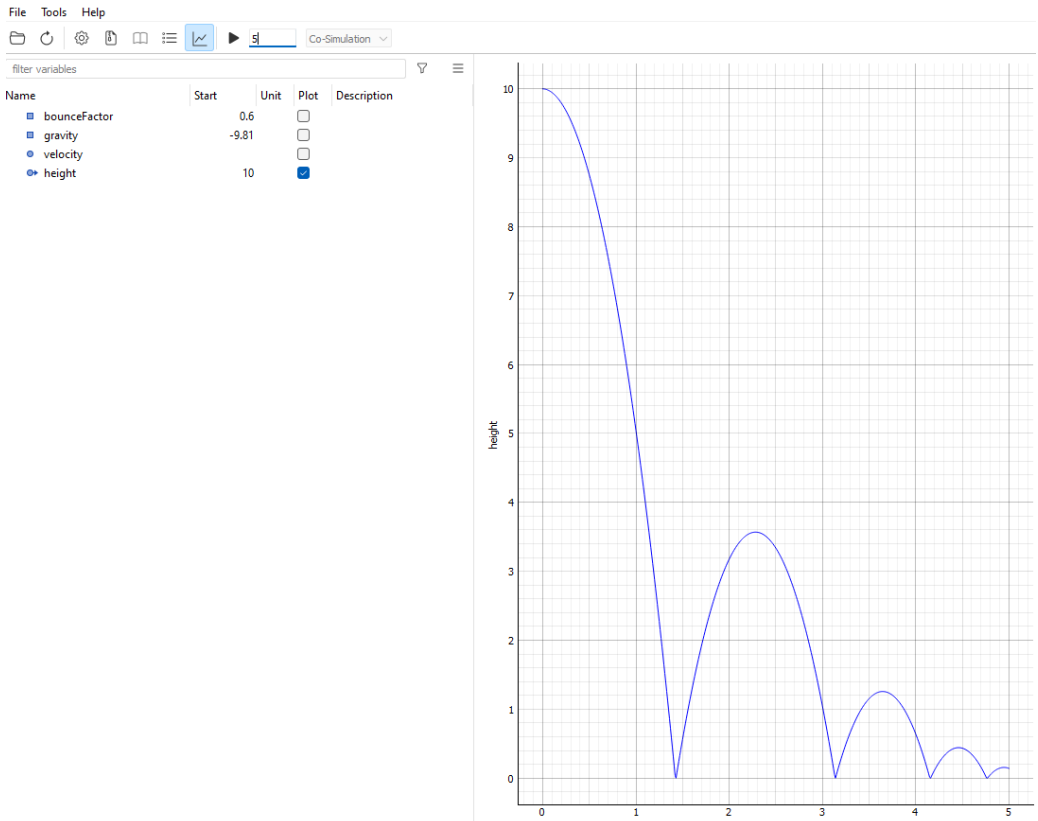

Figure 1: Bouncing ball model created with FMU4cpp simulated with FMPy.

Statement of need

A survey of existing open-source software that facilitates exporting FMUs from source-code is shown in Table 1.

Table 1: Survey of existing open-source FMI exporters.

Name	Language	Most recent Update
FMU4j (Hatledal, 2021)	Java	2022
JavaFMI (Siani, 2015)	Java	2021
cppfmu (Viproma, 2017)	C++	2024
Reference FMUs (Modelica Association, 2023)	C	2024
PythonFMU (Hatledal et al., 2020)	Python	2024
unifmu (Legaard et al., 2021)	Multiple	2024

The non-native solutions listed here provide similar capabilities as FMU4cpp, however the native solutions require a substantial expert knowledge and users are required to manually generate XML metadata as well as packaging the necessary files into an FMU. Thus, FMU4cpp provides a significant usability improvement over these alternatives as it: 1. Provides a simplified C++ based API to implement. 2. Generates the required XML metadata. 3. Verifies the FMU through automated compliance tests. 4. Cross-compiles the shared library. 5. Packages the FMU.

A framework like FMU4cpp is useful for generating both simple and more complex models that does not fit with the restrictions of a specialized modelling tools. Targeting C++ is also useful

compared to similar solutions for Python and Java, as a runtime is not necessarily required to invoke the model, increasing portability.

Future of FMU4cpp

With FMI 3.0 (Junghanns et al., 2021) released in 2022, an obvious and planned enhancement to FMU4cpp is to support this newer version of the standard. While version 2.0 of the standard is still the most widely supported by importing tools, adding support through FMU4cpp would further drive adoption and accommodate more effective simulations.

References

Blochwitz, T., Otter, M., Åkesson, J., Arnold, M., Clauss, C., Elmqvist, H., Friedrich, M., Junghanns, A., Mauss, J., Neumerkel, D., & others. (2012). Functional mockup interface 2.0: The standard for tool independent exchange of simulation models. *9th International Modelica Conference*, 173–184.

Blochwitz, T., Otter, M., Arnold, M., Bausch, C., Clauß, C., Elmqvist, H., Junghanns, A., Mauss, J., Monteiro, M., Neidhold, T., & others. (2011). The functional mockup interface for tool independent exchange of simulation models. *Proceedings of the 8th International Modelica Conference*, 105–114.

CATIA Systems. (2017). FMPy - a python library to simulate functional mock-up units (FMUs). In *GitHub repository*. https://github.com/CATIA-Systems/FMPy; GitHub.

Hatledal, L. I. (2021). *FMU4j - a JVM framework for creating FMUs*. https://github.com/Ecos-platform/FMU4j

Hatledal, L. I., Collonval, F., & Zhang, H. (2020). Enabling python driven co-simulation models with pythonfmu. *Proceedings of the 34th International ECMS-Conference on Modelling and Simulation-ECMS 2020*.

Junghanns, A., Gomes, C., Schulze, C., Schuch, K., Pierre, R., Blaesken, M., Zacharias, I., Pillekeit, A., Wernersson, K., Sommer, T., & others. (2021). The functional mock-up interface 3.0-new features enabling new applications. *Modelica Conferences*, 17–26.

Legaard, C. M., Tola, D., Schranz, T., Macedo, H. D., & Larsen, P. G. (2021). A universal mechanism for implementing functional mock-up units. *SIMULTECH*, 121–129.

Modelica Association. (2023). *Reference FMUs*. https://github.com/modelica/Reference-FMUs

Siani. (2015). *JavaFMI - a java library for working with FMUs*. https://bitbucket.org/siani/javafmi/wiki/Home

Viproma. (2017). *Cppfmu - a c++ library for FMU development*. https://github.com/viproma/cppfmu

DRAFT